Review retrospective: *the additively-separable 2D factorisation root, and an import that earns its keep transitively*

Claude Opus 4.8 (1M ctx), reviewer GPT-5.5 Pro

2026-06-05

Abstract

A soundness review of `Laplace/TwoD/AddSeparable.lean`: the generic factorisation infrastructure for additively-separable potentials $L(x, y) = U(x) + V(y)$ — partition-function and separable-observable factorisation, Gibbs-expectation factorisation, and the mixed-covariance vanishing — that the ten `KthKth*`/`QuarticSextic*` asymptotic files factor through. Result: **a clean fidelity pass with two dead-code removals** (a dead `open Real` and a literal no-op rewrite). The instructive moment was a *rejection*: both the reviewer and the reviewer-of-the-reviewer flagged the `SemiDegenerate` import as unused, and the full build refuted both — it is the transitive source of Mathlib's product integral.

1 Setting

This file is the root of the 2D "separable-lift" pattern documented in the seabed `CLAUDE.md`: every `Laplace/TwoD/KthKth*` and `QuarticSextic*` asymptotic expresses a 2D quantity (partition function, numerator, Gibbs moment) as a product of two 1D ones via the three factorisation theorems proved here. It was selected as a never-reviewed foundation sitting below ten consumers — the same "review the root" logic that took the modes-lean `SingularPair` and laplace `TailBound` reviews down their dependency chains.

2 What was reviewed

Seven public declarations: the potential `addSeparable U V z = U z.1 + V z.2`; its `simp` unfolding and the Boltzmann decomposition $e^{-t(U+V)} = e^{-tU}e^{-tV}$; and the four headlines — `partitionFunction_addSeparable_factor` ($Z_{2D} = Z_U Z_V$), `integral_separable_addSeparable` ($\int f \otimes g\, e^{-tL} = (\int f e^{-tU})(\int g e^{-tV})$), `gibbsExpectation_separable_addSeparable` ($\langle f \otimes g\rangle = \langle f\rangle_U \langle g\rangle_V$ when both $Z \neq 0$), and `gibbsCov_addSeparable_fst_snd_eq_zero` (mixed covariance vanishes). The corpus was the file docstrings, the `CLAUDE.md` architecture section, the deliberation tide-log, and `Laplace.Gibbs`.

3 Fidelity / soundness findings

Sound; no formula-level mismatch. The three factorisations are the expected ones (all three reduce to Mathlib's `integral_prod_mul` after the Boltzmann split), and the covariance proof follows the advertised pattern: $\langle fg\rangle = \langle f\rangle_U \langle g\rangle_V$, then the two 2D marginal expectations collapse to the

1D ones (instantiate $g = 1$, resp. $f = 1$, and use `gibbsExpectation_const`), so the covariance is $\langle f\rangle_U\langle g\rangle_V - \langle f\rangle_U\langle g\rangle_V = 0$.

The reviewer raised three *docstring-completeness* notes: the headlines for the Gibbs-expectation and covariance theorems state only the salient hypothesis ($Z_U, Z_V \neq 0$) and elide the integrability hypotheses the Lean statements also carry. These are recorded **report-only**: the docstrings are accurate-but-terse, the integrability is standard regularity fully visible in the signature, and enumerating every side-condition into a headline is the author's stylistic call — distinct from an *inaccurate* docstring, which a review does fix. Relatedly, the underscore-prefixed hypotheses `_hU`,`_hV`,`_hf`,`_hg` are genuinely inert (Mathlib's `integral_prod_mul` is unconditional) but are part of the documented API the ten consumers pass; removing them is a signature change, out of scope.

4 Simplifications applied

Two dead-code removals. A dead `open Real` (every `Real.exp`/`Real.exp_add` is qualified; `open MeasureTheory` stays — bare `Integrable` and $\int$ need it), and a literal no-op rewrite at the head of the covariance proof, `rw [show (fun z => f z.1 * g z.2) = (...same ...) from rfl]`, which rewrites an expression to itself and does nothing (the subsequent factorisation `rw` matches without it). Module green at 2699 jobs, no warnings (baseline had none either), public signatures byte-identical. GPT affirmed.

5 Disagreements and open questions

The reviewer's top simplification — "drop the dead `import Laplace.TwoD.SemiDegenerate`" — was **rejected by the full build**. Removing it makes `integral_prod_mul` an unknown identifier, with a cascade of application-mismatch and "invalid projection: index 2" errors: `SemiDegenerate` is the (indirect) source of Mathlib's product-integral API and the $\mathbb{R}\times\mathbb{R}$ product-measure structure, neither of which `Laplace.Gibbs` pulls in. The "no `SemiDegenerate` symbol is referenced" check — true at the symbol level — missed the *transitive Mathlib* dependency. There is a real architectural oddity underneath, though: depending on a 514-line case-by-case sibling for a Mathlib lemma is backwards (the docstring itself says the dependency should eventually point the other way). The clean fix is to import the Mathlib product-integral file directly — but that is a structural import-swap, a forward tide, not a review edit.

6 Lessons

- **"References no symbol from X" does not imply "X is a dead import".** An import can be load-bearing purely *transitively* — here `SemiDegenerate` is the only path to Mathlib's `integral_prod_mul`. The symbol-grep is a hypothesis; the *build* is the verdict. This is the import-side analogue of the per-file dead-`open` rule, and it caught a finding that two independent models (the reviewer, and my own pre-read) both got wrong.
- **A literal `rw [show A = A from rfl]` is a clean, silent dead-code class.** It rewrites to itself, so removal is statement-preserving by construction; unlike an `unusedTactic`-flagged no-op it raises no warning either way, so the rebuild is the only check needed.
- **Accurate-but-terse $\neq$ inaccurate.** The line between a landable docstring fix and a report-only note is whether the prose is *wrong* (fix it) or merely *incomplete on plumbing* (report it); a headline stating the salient hypothesis and eliding standard regularity is the latter.

7 Follow-ups

- **Forward tide (architectural):** replace `import Laplace.TwoD.SemiDegenerate` with a direct import of the Mathlib product-integral file (`Mathlib.MeasureTheory.Constructions.Prod.Integral` or wherever `integral_prod_mul` lands), removing the backwards dependency on the 514-line case-by-case file. Confirm the ten consumers still build.
- **Forward tide (proof golf):** the reviewer's `simpa ...using integral_prod_mul` collapses and the `simpa [mul_one]/[one_mul]` rewrites for the marginal-collapse subproofs would shorten the file; declined here as risk-bearing restructuring outside the review's safe class.
- Next review surface: the `TwoD/KthKth*` and `QuarticSextic*` consumers (now reviewable with confidence in their factorisation root), `OneD/NearDegenerate`, `OneD/UniversalAsymptotics`, and the large `Multi/Covariance*` trio.